

Coding Style - Simple Hinglish Summary

Source: <https://javascript.info/coding-style>

Main baat

Code **jitna ho sake saaf aur padhne me aasan** hona chahiye. Programming ki asli kala yahi hai: complex kaam ko aise code karna jo **sahi bhi ho aur insaan ke liye readable bhi**. Achhi coding style isme bahut madad karti hai.

Dhyan: Yahan koi "zaroor karna hi hai" wale rules nahi hain. Ye **style preferences** hain, koi dharmik niyam nahi.

1. Syntax

Curly Braces

Zyadatar JS projects me "**Egyptian**" style use hoti hai: opening brace **usi line par** keyword ke saath, nayi line par nahi. Brace se pehle **space** hona chahiye.

```
javascript

if (condition) {
  // do this
  // ...and that
  // ...and that
}
```

Single-line construct (jaise `if (condition) doSomething()`) ka sawal: braces lagayein ya nahi? Ye 4 variants dekho:

1. 😞 Beginners aisa karte hain. **Bura**. Braces ki zarurat nahi:

```
javascript

if (n < 0) {alert(`Power ${n} is not supported`);}
```

2. 😞 Alag line par bina braces ke. **Kabhi mat karo**, naya line add karte waqt galti hona aasan hai:

```
javascript

if (n < 0)
  alert(`Power ${n} is not supported`);
```

3. 😊 Ek line me bina braces ke. **Chalta hai**, agar chhota ho:

javascript

```
if (n < 0) alert(`Power ${n} is not supported`);
```

4. 😊 Sabse best:

javascript

```
if (n < 0) {  
  alert(`Power ${n} is not supported`);  
}
```

Bahut chhote code ke liye ek line allowed hai, jaise `if (cond) return null`. Lekin code block (4th variant) aam taur par zyada readable hota hai.

Line Length

Kisi ko bahut lambi horizontal line padhna pasand nahi. Unhe **tod do**.

javascript

```
// backticks se string ko kai lines me tod sakte hain  
let str = `  
  ECMA International's TC39 is a group of JavaScript developers,  
  implementers, academics, and more, collaborating with the community  
  to maintain and evolve the definition of JavaScript.  
`;
```

`if` statements ke liye:

javascript

```
if (  
  id === 123 &&  
  moonPhase === 'Waning Gibbous' &&  
  zodiacSign === 'Libra'  
) {  
  letTheSorceryBegin();  
}
```

Max line length team me tay honi chahiye, aam taur par **80 ya 120 characters**.

Indents

Do tarah ke indents:

Horizontal indents: 2 ya 4 spaces. Spaces ya Tab key se ho sakta hai. Kaunsa chunein, ye purani bahas hai. Aajkal **spaces zyada common** hain. Spaces ka fayda: zyada flexible configuration, jaise parameters ko opening bracket se align karna:

```
javascript

show(parameters,
    aligned, // 5 spaces padding
    one,
    after,
    another
) {
    // ...
}
```

Vertical indents: khali lines se code ko logical blocks me todna. Ek function ko bhi aksar blocks me baant sakte hain. Neeche variables ka initialization, main loop aur result return alag alag hain:

```
javascript

function pow(x, n) {
    let result = 1;
    // <--
    for (let i = 0; i < n; i++) {
        result *= x;
    }
    // <--
    return result;
}
```

Jahan readability badhe wahan extra newline daalo. **9 lines se zyada code bina vertical indent ke nahi hona chahiye.**

Semicolons

Har statement ke baad semicolon **lagao**, chahe skip ho sakta ho. JS me aisi jagah hain jahan line break semicolon nahi maana jata, aur code galti ke liye vulnerable ho jata hai.

Experienced ho to **no-semicolon style** (jaise StandardJS) chun sakte ho. Warna semicolon lagana hi behtar hai. **Zyadatar developers lagate hain.**

Nesting Levels

Code ko **bahut zyada gehra nest** karne se bacho. Loop me `continue` se extra nesting bacha sakte ho.

Aisa likhne ki jagah:

javascript

```
for (let i = 0; i < 10; i++) {  
  if (cond) {  
    ... // <- ek aur nesting level  
  }  
}
```

Aisa likho:

javascript

```
for (let i = 0; i < 10; i++) {  
  if (!cond) continue;  
  ... // <- extra nesting nahi  
}
```

`if/else` aur `return` ke saath bhi aisa hi kar sakte ho. Ye dono constructs same hain:

Option 1:

javascript

```
function pow(x, n) {  
  if (n < 0) {  
    alert("Negative 'n' not supported");  
  } else {  
    let result = 1;  
  
    for (let i = 0; i < n; i++) {  
      result *= x;  
    }  
  
    return result;  
  }  
}
```

Option 2 (behtar):

javascript

```
function pow(x, n) {
  if (n < 0) {
    alert("Negative 'n' not supported");
    return;
  }

  let result = 1;

  for (let i = 0; i < n; i++) {
    result *= x;
  }

  return result;
}
```

Dusra version zyada readable hai kyunki `(n < 0)` wala "special case" pehle hi handle ho jata hai. Uske baad **main code flow** bina extra nesting ke chalta hai.

2. Function Placement

Kai "helper" functions aur unhe use karne wala code likh rahe ho, to **teen tarike** hain:

1. Functions upar, code neeche:

```
javascript

// function declarations
function createElement() { ... }
function setHandler(elem) { ... }
function walkAround() { ... }

// code jo unhe use karta hai
let elem = createElement();
setHandler(elem);
walkAround();
```

2. Code pehle, functions baad me:

```
javascript
```

```
// code jo functions use karta hai
let elem = createElement();
setHandler(elem);
walkAround();

// --- helper functions ---
function createElement() { ... }
function setHandler(elem) { ... }
function walkAround() { ... }
```

3. Mixed: function wahan declare hota hai jahan pehli baar use hota hai.

Zyadatar doosra variant preferred hai. Kyunki code padhte waqt hum pehle jaanna chahte hain ki wo kya karta hai. Code pehle ho to shuru se hi clear ho jata hai. Phir shayad functions padhne ki zarurat hi na pade, khaaskar agar unke naam descriptive hon.

3. Style Guides

Style guide me "code kaise likhna hai" ke general rules hote hain: kaunse quotes, kitne spaces ka indent, max line length, aur bahut se chhote rules.

Team ke saare members ek hi style guide use karein to code **uniform** dikhta hai, chahe kisi ne bhi likha ho. Apna style guide banane ki aam taur par zarurat nahi, kai existing guides hain.

Popular choices:

- Google JavaScript Style Guide
- Airbnb JavaScript Style Guide
- Idiomatic.JS
- StandardJS

Naye developer ho to **is chapter ke cheat sheet se shuru karo**, phir dusre guides dekhkar decide karo ki kaunsa pasand aata hai.

4. Automated Linters

Linters aise tools hain jo tumhare code ki **style automatically check** karte hain aur sudhar ke suggestions dete hain.

Bada fayda: style checking se kuch **bugs** bhi pakde jate hain, jaise variable ya function ke naam me typo. Isliye linter use karna recommended hai, chahe kisi khaas style ko na follow karna ho.

Well-known linters:

- **JSLint:** sabse pehle linters me se ek
- **JSHint:** JSLint se zyada settings

- **ESLint:** shayad sabse naya (author ye use karta hai)

Zyadatar linters popular editors ke saath integrate hote hain: bas editor me plugin enable karo aur style configure karo.

ESLint ke liye steps:

1. **Node.js** install karo.
2. `npm install -g eslint` se ESLint install karo.
3. Project ke root me `.eslintrc` naam ki config file banao.
4. Apne editor me ESLint ka plugin install/enable karo.

`.eslintrc` ka example:

```
json
{
  "extends": "eslint:recommended",
  "env": {
    "browser": true,
    "node": true,
    "es6": true
  },
  "rules": {
    "no-console": 0,
    "indent": 2
  }
}
```

`"extends"` ka matlab config "eslint:recommended" settings par based hai, uske baad apni settings likhte hain. Web se style rule sets download karke bhi extend kar sakte ho. Kuch IDEs me built-in linting bhi hoti hai, jo convenient hai lekin ESLint jitni customizable nahi.

Summary

Is chapter (aur style guides) ke saare syntax rules ka maqsad code ki **readability** badhana hai. Sab debatable hain.

"Behtar" code likhne ke liye khud se ye sawal poochho:

- "Kya cheez code ko zyada readable aur samajhne me aasan banati hai?"
- "Kya cheez mujhe galtiyan se bachne me madad karegi?"

Popular style guides padhne se style trends aur best practices ka pata chalta rehta hai.

Practice Task (Answer)

Bad style wale code me kya galat hai?

javascript

```
function pow(x,n)
{
    let result=1;
    for(let i=0;i<n;i++) {result*=x;}
    return result;
}

let x=prompt("x?", ''), n=prompt("n?", '')
if (n<=0)
{
    alert(`Power ${n} is not supported, please enter an integer number greater th
}
else
{
    alert(pow(x,n))
}
```

Galtiyan:

- Arguments ke beech **space nahi** (`(x,n)`)
- Curly brace **alag line par** hai
- `=`, `<`, `++` ke aas-paas **spaces nahi**
- `{ ... }` ke andar ka content **nayi line par** hona chahiye
- Ek line me do variables (`let x=..., n=...`) aur `;` **missing**
- `if (n<=0)` me spaces nahi aur upar ek khali line honi chahiye
- Lambi line ko kai lines me todna chahiye
- `pow(x,n)` me space nahi aur `;` missing

Sahi version:

javascript

```
function pow(x, n) {
  let result = 1;

  for (let i = 0; i < n; i++) {
    result *= x;
  }

  return result;
}

let x = prompt("x?", "");
let n = prompt("n?", "");

if (n <= 0) {
  alert(`Power ${n} is not supported,
  please enter an integer number greater than zero`);
} else {
  alert( pow(x, n) );
}
```

Quick Summary

Topic	Rule
Braces	Opening brace usi line par, <code>{ }</code> hamesha lagao
Line length	80 ya 120 characters se zyada nahi
Indent	2 ya 4 spaces
Vertical indent	9 lines se zyada bina khali line ke nahi
Semicolon	Har statement ke baad lagao
Nesting	<code>continue</code> aur early <code>return</code> se kam karo
Function placement	Pehle code, phir helper functions (zyadatar)
Style guide	Google, Airbnb, StandardJS me se chuno
Linter	ESLint use karo, typos aur bugs bhi pakadta hai